

Contour — catchment estimation and pond siting for a village

A web application and API that takes a contour survey, or an area selected on the map, reconstructs the terrain, delineates the catchment above each candidate pond site, and returns the pond location, the catchment area and the water it can collect — as JSON, and overlaid on the map.

SUBMITTED BY	Rajeev Kumar · Roll 12341700
GITHUB REPOSITORY	https://github.com/rajeev-sr/pond-track
WEB APPLICATION	http://10.1.75.53:3274
API ROUTE	POST http://10.1.75.53:3274/api/v1/analyzeContour POST http://10.1.75.53:3274/api/v1/findCatchment (alias) POST http://10.1.75.53:3274/api/v1/analyzeArea (an area selected on the map)
API DOCUMENTATION	http://10.1.75.53:3274/docs (Swagger UI) · /redoc · /openapi.json
DEMONSTRATED ON	<code>contours_1m.kml</code> — 1,355 contour lines, 32 levels at 1.0 m, relief 31.0 m

Contents against the brief

REQUIRED	PROVIDED	SECTION
✓ GitHub repository	https://github.com/rajeev-sr/pond-track	§1
✓ Working API route URL	POST http://10.1.75.53:3274/api/v1/analyzeContour	§2
✓ Catchment estimation approach	Method, stage by stage, with the formulae	§3
✓ Demonstration on the provided map	Request, response and screenshots	§4
✓ Select a land area on the map	Draw a rectangle, or one click for the sample area	§5
✓ Pond location, catchment area and water collected for that area	The <code>summary</code> block and every candidate	§5
✓ Results overlaid on the map	Volume on the marker, catchment labelled, popup with all three	§5
✓ Fast and functional on the lab systems	Gateway, one analysis per system, measured under load	§6
✓ API documentation	Every route, schemas and the interactive spec	§7

1 · Repository

The full source, including the test suite, the deployment compose file and the design documents, is at:

```
https://github.com/rajeev-sr/pond-track
```

The tree is laid out so each concern can be read on its own:

PATH	HOLDS
backend/app/api/v1/	HTTP routes, request validation, problem responses
backend/app/services/	The domain: interpolation, hydrology, siting, pond design
backend/app/providers/	External data — elevation, land cover, soil, rainfall, OSM
backend/app/tests/	Unit, property, golden, integration and real-browser tests
frontend/src/	React map interface that consumes the same API

Running it

```
git clone https://github.com/rajeev-sr/pond-track.git
cd pond-track
cp .env.example .env
make up          # postgres, redis, api, tiler, frontend
make demo        # analyse the bundled contour map end to end
```

Nothing needs to be registered for or paid for: every data source the service reads is openly licensed and keyless.

2 · Working API route

The route named in the brief is implemented under both names it may be looked for:

METHOD	URL	PURPOSE
POST	<code>http://10.1.75.53:3274/api/v1/analyzeContour</code>	Upload a contour map; receive ranked sites with their catchments
POST	<code>http://10.1.75.53:3274/api/v1/findCatchment</code>	Alias of the above, byte-for-byte the same response
POST	<code>http://10.1.75.53:3274/api/v1/analysis</code>	The same work as a background job, for large sheets
POST	<code>http://10.1.75.53:3274/api/v1/analyzeArea</code>	No file: a rectangle selected on the map, analysed on Copernicus 30 m terrain (\$5)
POST	<code>http://10.1.75.53:3274/api/v1/analysis/area</code>	The same as a job — what the web application uses

Request

`multipart/form-data`. Only the map is required, as `contour_map` (or its alias `file`).

FIELD	TYPE	DEFAULT	MEANING
contour_map	upload	—	The contour map , <code>.kml</code> / <code>.kmz</code> / <code>.xml</code> . This is the field name to use.
file	upload	—	Accepted alias for <code>contour_map</code> , for callers that already send this name. Send one or the other, not both.
max_sites	int 1-25	5	How many ranked sites to return
max_slope_pct	float	8.0	Slope above which ground is not buildable
cell_size_m	float	auto	Grid resolution; derived from contour spacing if omitted
enrich	bool	true	Fetch soil, land cover and rainfall
include_contours	bool	false	Return the parsed contours as GeoJSON

Verification

The transcript in §4 is a real call against a running instance; the response was 217,001 bytes of JSON returned in 10.61 s.

The response quoted in §4 was measured against the URL above — that instance answering, not a local run. The captures behind §5 and §6 each name their own source beneath them. Readiness at the time of capture (Redis is optional: without it, a job is kept by the API process that runs it, and the gateway sends each browser back to that process):

```
$ curl http://10.1.75.53:3274/api/v1/health/ready
{"status": "ready", "checks": {"database": "ok", "redis": "not_configured"}}
```

Endpoints verified on the deployed instance

ENDPOINT	METHOD	RESULT
/api/v1/analyzeContour with contour_map	POST	200 · 212 KB
/api/v1/findCatchment	POST	200 · 212 KB
/api/v1/terrain/contour-map	POST	200
/api/v1/terrain/contours	POST	200 · 670 KB
/api/v1/terrain/derivatives	POST	200 · 3 KB
/api/v1/hydrology/streams	POST	200 · 245 KB
/api/v1/hydrology/catchment	POST	200 · 30 KB
/api/v1/land/available	POST	200 · 132 parcels
/api/v1/land/cadastral	POST	200
/api/v1/analysis → status → result → export	POST/GET	202 → done → 200 → 200
/api/v1/suitability/analyze → sites → compare	POST/GET	202 → 200 → 200
/api/v1/reports/generate → download	POST/GET	201 → 200 · 102 KB PDF
/api/v1/analyzeArea	POST	200 · 115 KB
/api/v1/terrain/{dem_id}/overlays → overlay	GET	200 → 200 · 200 (2 PNG)
/api/v1/analysis/area → status	POST/GET	202 → done
/api/v1/health/features	GET	200 · village search True
/api/v1/villages/search	GET	200 · 3 results
/api/v1/villages/{id}/boundary	GET	200 · 10 KB
/docs · /redoc · /openapi.json	GET	200 · 200 · 200

Malformed requests are rejected with a problem document rather than a stack trace: no file → 400 naming contour_map; both field names → 400; a non-contour extension → 400; a .kml that is not XML → 422; an unknown job id → 404.

Called by scripts/capture_endpoints.py on stu79 sys3 against http://10.1.75.53:3274, one browser id throughout, at 2026-09-26T21:50:52Z.

3 · How the catchment is estimated

A contour sheet gives elevation along lines and says nothing about the ground between them. Estimating a catchment from it is four steps: rebuild a continuous surface, make that surface drainable, work out where each cell's water goes, then collect every cell whose water reaches the point of interest.

3.1 A surface from the lines

Contour vertices are triangulated (Delaunay) and the surface interpolated linearly inside each triangle, giving a regular grid. On the demonstration sheet, 159,113 vertices from 1,355 lines became a 650 × 527 grid at 5.0 m — 342,550 cells.

The working CRS is chosen from the sheet's own longitude — UTM zone 32644 here — so every area, slope and distance below is in metres, not degrees. No location is fixed in the code.

Interpolation cannot invent detail. A hollow shallower than the contour interval (1.0 m here) cannot appear in the grid at all. This bounds the whole estimate and is reported rather than smoothed over.

3.2 Making the surface drainable

An interpolated surface contains closed depressions, some real and some artefacts. Water cannot leave them, so accumulation would stop there. They are raised to their spill level by **Priority-Flood with an ε gradient**, which leaves a shallow descending slope across the filled area instead of a dead flat one. Where a depression is caused by a road or bund crossing a channel, a bounded least-cost breach is cut instead — filling would erase a real channel.

3.3 Where each cell's water goes

Flow direction is assigned by **D8**: each cell drains to whichever of its eight neighbours lies steepest downhill. Accumulation is then counted by walking the cells in dependency order — every cell is visited once, after all cells that drain into it — so the count at a cell is the number of cells upstream of it. Multiplying by cell area converts that to a contributing area.

$$A = N \cdot c^2$$

A contributing area (m²), N cells upstream, c cell size. At 5.0 m one cell is 25 m², so the recommended site's 72,100 cells give 180.25 ha.

3.4 Collecting the catchment

The catchment above a point is every cell whose flow path reaches it. It is found by walking the flow graph upstream from the outlet. Two details decide whether the answer is meaningful:

- **The outlet is snapped to a channel.** A point a few metres off the channel sits on a hillside, and its catchment is a few hectares rather than a few hundred — plausible-looking and wrong. The requested point is moved to the nearest cell above the channel threshold, and the distance moved is reported.
- **A catchment touching the sheet edge is a lower bound**, because the contributing area continues beyond the survey. This is flagged per catchment (`touches_grid_edge`) rather than presented as a measurement. On the recommended site it is `False`.

3.5 What is reported about it

Area alone does not describe a catchment's behaviour, so the shape and response are derived too:

QUANTITY	VALUE	DERIVED FROM
Contributing area	180.25 ha	72,100 cells × cell area
Perimeter	8,340 m	Boundary of the traced region
Relief	16.01 m	Max – min elevation within it
Mean slope	3.94 %	Cell-wise gradient of the conditioned surface
Longest flow path	2,836.3 m	Traced along D8 directions
Time of concentration	65.1 min	Kirpich, from path length and slope
Form factor	0.2241	Area / longest path ²
Compactness	1.752	Perimeter vs an equal-area circle

$$T_c = 0.01947 \cdot L^{0.77} \cdot S^{-0.385}$$

Kirpich (1940). L is the longest flow path in metres, S its average gradient. A short, steep catchment concentrates quickly and needs a larger spillway for the same area.

3.6 The channel network the catchment sits in

A cell is treated as a channel once its contributing area passes a threshold — 1 ha by default, deliberately small, because a nala draining a few hectares is exactly the feature a village check dam sits on. On the demonstration sheet the network is 273 reaches totalling 50.25 km; within the recommended site's catchment it is 60 reaches, 12.23 km, a drainage density of 6.79 km/km².

Drainage density is quoted for the catchment, not the sheet. It is length per unit *basin* area; measured over the survey rectangle it would average unrelated catchments, so the API returns it for a delineated basin and withholds it otherwise.

3.7 From catchment to a pond proposal

The catchment is the input to everything that follows. Yield uses SCS-CN with the initial abstraction at 0.3S rather than 0.2S, following CWC and IMD practice for Indian catchments:

$$Q = (P - 0.3S)^2 / (P + 0.7S), \quad S = 25400/CN - 254$$

P, Q in mm; Q = 0 for P ≤ 0.3S. Curve number from hydrologic soil group and land cover — here.

Storage comes from the surface itself: the pond is flood-filled level by level to give a stage-storage curve, with volume between levels by the prismoidal rule. Candidate sites are ranked over nine criteria weighted by AHP, and only over ground that survives the exclusion veto — existing water, rivers, buildings and roads.

4 · Demonstration on the provided contour map

The sheet bundled with the assignment, `contours_1m.kml` (6,553 KB), analysed end to end. Nothing below is typed by hand: it is read from the response this call returned.

4.1 The call

```
$ curl -X POST http://10.1.75.53:3274/api/v1/analyzeContour \
-F "contour_map=@contours_1m.kml" -F "max_sites=5"
```

HTTP 200 · 217,001 bytes · 10.61 s

4.2 What was read from the file

Contour lines parsed	1,355
Lines whose elevation could not be resolved	0
Distinct levels	32 at 1.0 m
Elevation range	267.0 – 298.0 m
Elevation source	placemark_name
Working CRS	EPSG:32644
Data tier	full

4.3 The recommended site and its catchment

Rank / kind	#1 · channel position	Longest flow path	2,836.3 m
Suitability	82.5 / 100	Time of concentration	65.1 min
Location	21.25114, 81.29545	Clipped by sheet edge	False
Catchment area	180.25 ha	Design depth	4.50 m
Relief	16.01 m	Gross capacity	81,682 m ³
Mean slope	3.94 %	Binding constraint	practical excavation depth

4.4 All ranked candidates

#	KIND	SCORE	CATCHMENT (HA)	TC (MIN)	CAPACITY (M ³)
1	channel position	82.5	180.2	65.1	81,682
2	natural depression	69.0	97.5	43.2	81,682
3	natural depression	68.0	112.2	48.7	3,119
4	channel position	67.7	18.6	20.6	15,319
5	channel position	67.0	26.6	25.0	22,309

4.5 Ground excluded before ranking

106,461 cells were vetoed before any site was scored, from OpenStreetMap, land cover, terrain — reported confidence **high**.

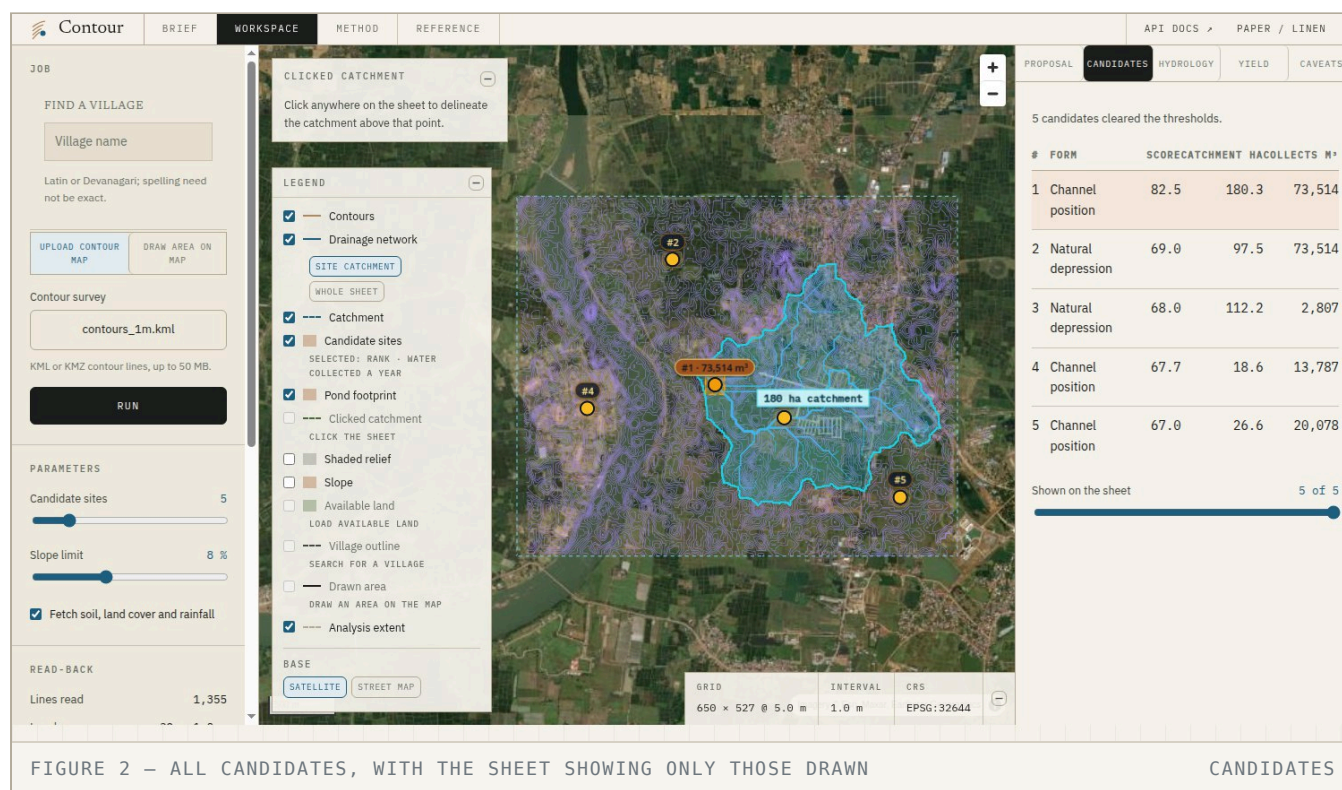
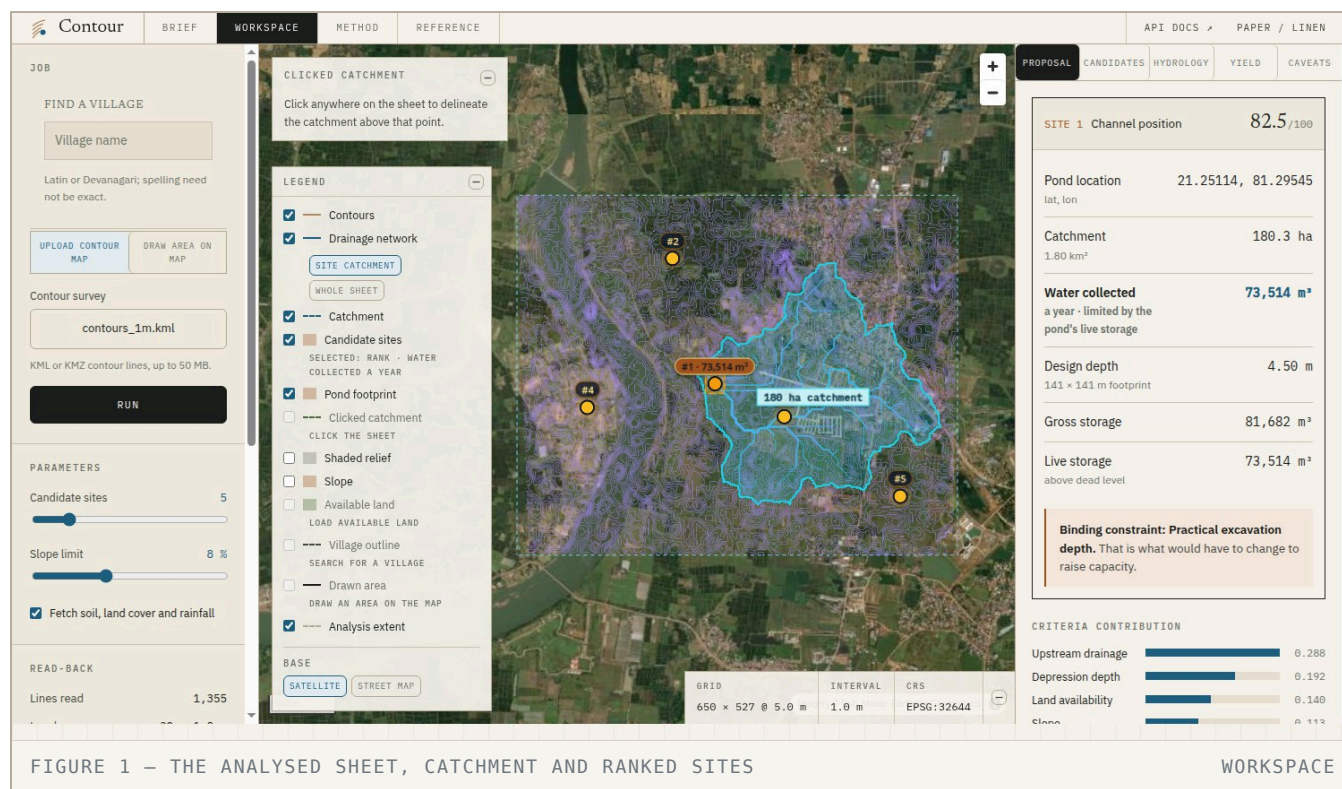
RULE	CELLS
standing water	2,757
major watercourse	53,843
building	1,453
road	45,694
land cover water	36,597
land cover built up	32,266

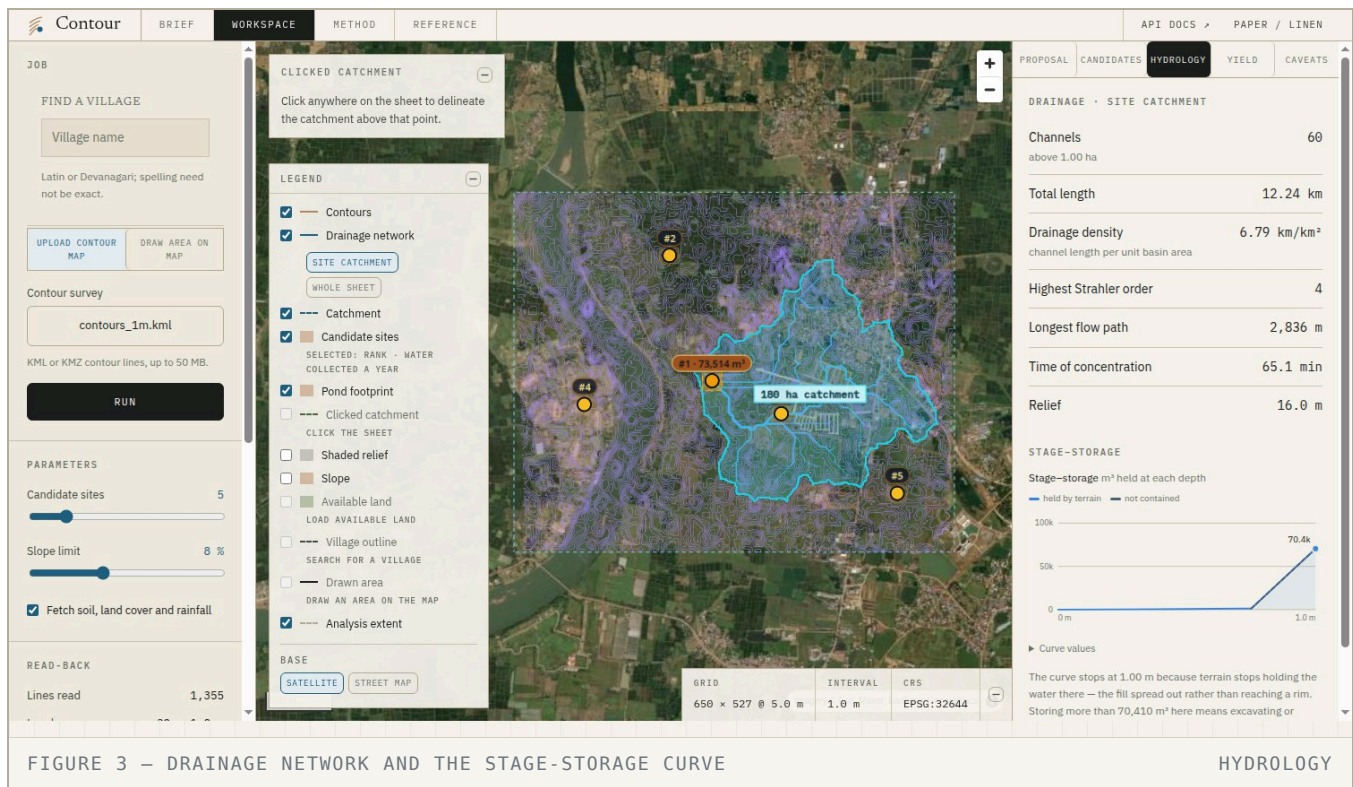
4.6 Where the time went

STAGE	SECONDS
parse	0.372
interpolate	6.126
condition	1.154
flow_routing	0.678
enrichment	0.171
siting	0.265
catchments	1.721

Data layers used: elevation, slope, flow_accumulation, depression_depth, land_use_land_cover, soil_hydrologic_group, rainfall, reference_evapotranspiration. No layer was unavailable.

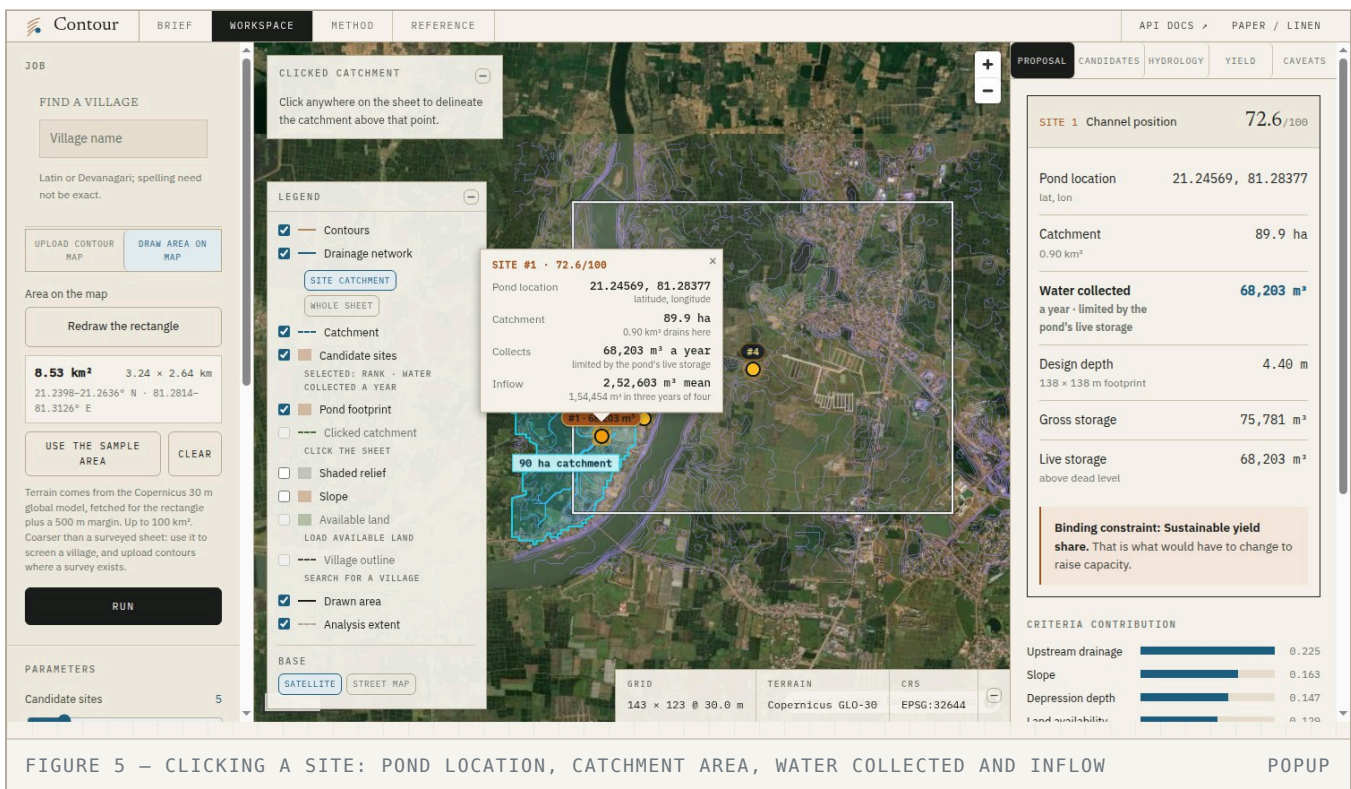
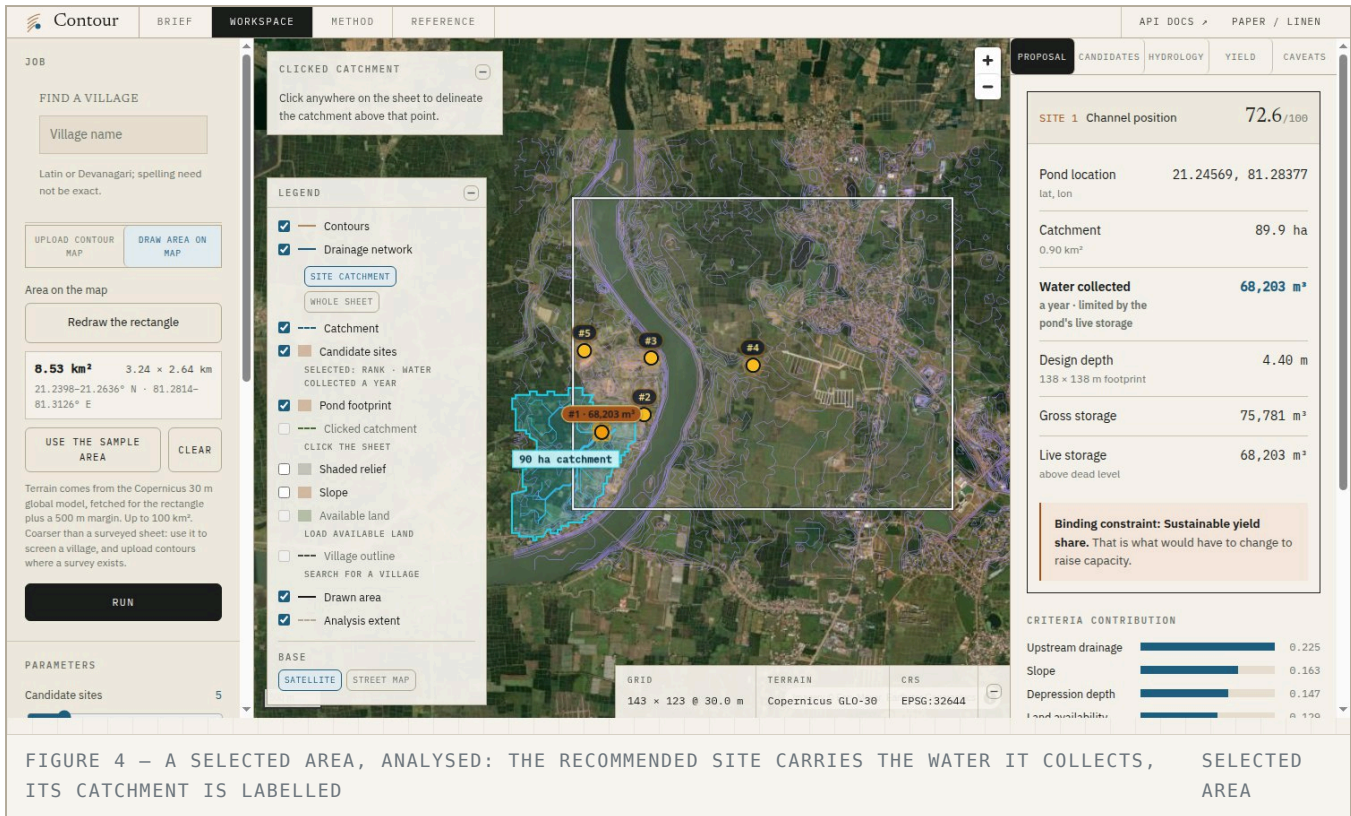
4.7 The same analysis in the browser





5 · Selecting an area on the map

A village without a contour survey can still be assessed. In the workspace, **Draw area on map** turns a drag on the map into a rectangle; its size is shown while it is drawn, and **Run** refuses anything under 0.1 km² or over 100 km² before a request is made. **Use the sample area** draws the sample sheet's own extent in one click. The terrain comes from the Copernicus GLO-30 elevation model, read for the rectangle plus a 500 m margin — water reaches a site from beyond the line that was drawn — and sites are proposed only inside the rectangle. From there every stage is the pipeline of \$3.



5.1 The same through the API

```
curl -X POST http://10.1.75.53:3274/api/v1/analyzeArea \
-H 'Content-Type: application/json' \
-d '{"bbox": [81.2814, 21.2398, 81.3126, 21.2636]}'
```

The response has the shape of a contour analysis, with `contour_map` null and `terrain_source` naming Copernicus DEM GLO-30 at 30 m. It opens with the three results:

RESULT	VALUE	BASIS
Pond location	21.24569, 81.28377	Site 1, suitability 72.6/100
Catchment area	89.9 ha	0.90 km² draining to the site, delineated by D8 on the conditioned surface
Water collected	68,203 m³ a year	the pond's live storage, capped by the catchment's 75 % dependable annual inflow: the water it can collect in three years of four; limited by storage. Inflow 252,603 m³ mean, 154,454 m³ in three years of four

Every candidate carries the same figure for itself. They are never added up: catchments nest, so a total would count the same water twice.

#	POND LOCATION	CATCHMENT	COLLECTS LIMITED BY
1	21.24569, 81.28377	89.9 ha	68,203 m³ storage
2	21.24703, 81.28724	118.1 ha	72,743 m³ storage
3	21.25137, 81.28783	118.1 ha	72,743 m³ storage
4	21.25081, 81.29621	270.1 ha	72,743 m³ storage
5	21.25192, 81.28233	37.1 ha	28,187 m³ storage

Captured from the stu79 deployment, <http://10.1.75.53:3274>, requested from inside the lab network (sys3) on 2026-09-27, after one warm-up run.

5.2 How much 30 m terrain costs

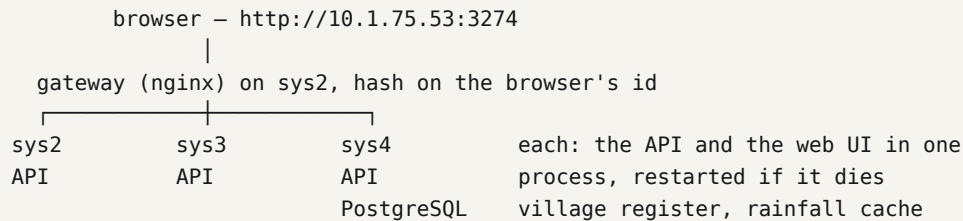
Over the sample sheet's own extent, the same land analysed both ways (8.53 km², enrichment off so only the terrain differs):

	CONTOUR UPLOAD	SELECTED AREA
Grid	650 × 527 at 5 m	143 × 123 at 30 m, with the margin
Relief	31.0 m	30.1 m
Recommended site	score 84.0 · 264 ha	score 67.5 · 270 ha

Elevations agree at $r = 0.905$ over 3,327 points, with a constant offset of -5.0 m and 2.3 m of spread about it. But the sites differ: no 30 m candidate lies within 432 m of the contour analysis's recommended site, because each cell covers 36 times the ground. So a selected area is a screening tool for any village; where a survey exists, the contour upload is the answer to build on. The response says as much in `terrain_source.note`.

6 · Performance, stress and scaling

The service runs on three lab systems, each with **512 MB of memory and one CPU** (the fourth allocated system, sys1, has not been reachable). One analysis of the sample sheet peaks at a few hundred megabytes, so the design starts from one rule: **one analysis at a time per system**, and never a crash in place of an answer.



MECHANISM	WHAT IT DOES
Overload guard	One analysis per system. A direct API call beyond that is answered at once with <code>503</code> and <code>Retry-After</code> ; a run from the web application waits in a queue and says so.
Affinity	The page sends a per-browser id, and the gateway hashes on it, so a user's follow-up calls reach the system holding their terrain. Not the client address: every user on one lab subnet would land on one system.
Health	An instance that stops answering is taken out of rotation; its users move to the next one and come back when it restarts.
Memory	GDAL's cache and glibc's arenas bounded (the first sized itself from the host's RAM); terrain grids kept on disk rather than in memory; finished jobs capped.
Upstream data	Terrain, soil, land cover, rainfall and OpenStreetMap all cached on disk; one request per OpenStreetMap window at a time, and a refused window remembered for ten minutes.
The lab's internet	From the lab systems, S3, NASA POWER and SoilGrids time out and Open-Meteo answers 429, so the 26 terrain tiles covering Chhattisgarh are kept on each system and 30 years of NASA POWER daily rainfall for its 65 cells are in the database. Open-Meteo is left alone for an hour after a refusal, and the rainfall step returns at its deadline with whichever source answered.

6.1 Under load

Users placed deliberately: one on each system, two on each, or all on one. Web-application runs of the sample area (queued, polled each second):

SCENARIO	USERS	OK	503	P50	P95	FOLLOW-UPS
single	1	2/2	0	1.0 s	1.0 s	2/2
spread	3	6/6	0	1.0 s	7.1 s	6/6
doubled	6	12/12	0	3.1 s	3.1 s	12/12
collide	3	6/6	0	3.6 s	4.2 s	6/6

Process memory peaked at 194–277 MB per system of 512 (sampled each second; sys4's figure includes PostgreSQL), 0 OOM kills.

The same as direct API calls, where overload is answered rather than queued:

SCENARIO	USERS	OK	503	P50	P95	FOLLOW-UPS
single	1	1/1	0	1.1 s	1.1 s	1/1
spread	3	3/3	0	1.0 s	1.2 s	3/3
doubled	6	3/6	3	1.0 s	1.1 s	3/3
collide	3	1/3	2	0.9 s	0.9 s	1/1

Process memory peaked at 194–263 MB per system of 512 (sampled each second; sys4's figure includes PostgreSQL), 0 OOM kills.

Contour uploads of the sample sheet (342 550 cells, the heaviest input):

SCENARIO	USERS	OK	503	P50	P95	FOLLOW-UPS
single	1	1/1	0	10.9 s	10.9 s	1/1
spread	3	3/3	0	11.6 s	12.6 s	3/3

Process memory peaked at 344–403 MB per system of 512 (sampled each second; sys4's figure includes PostgreSQL), 0 OOM kills.

Three users on three systems are served as fast as one user on one. A fourth waits its turn in the web application, or is told to retry by the API, rather than taking a system down; every follow-up call reached its own terrain, and no system was OOM-killed.

6.2 One system failing

An instance was killed while a user pinned to it kept making requests: 0 of 44 requests failed; the user was served by another system at 1.5 s and back on their own at 21.8 s.

6.3 Limits

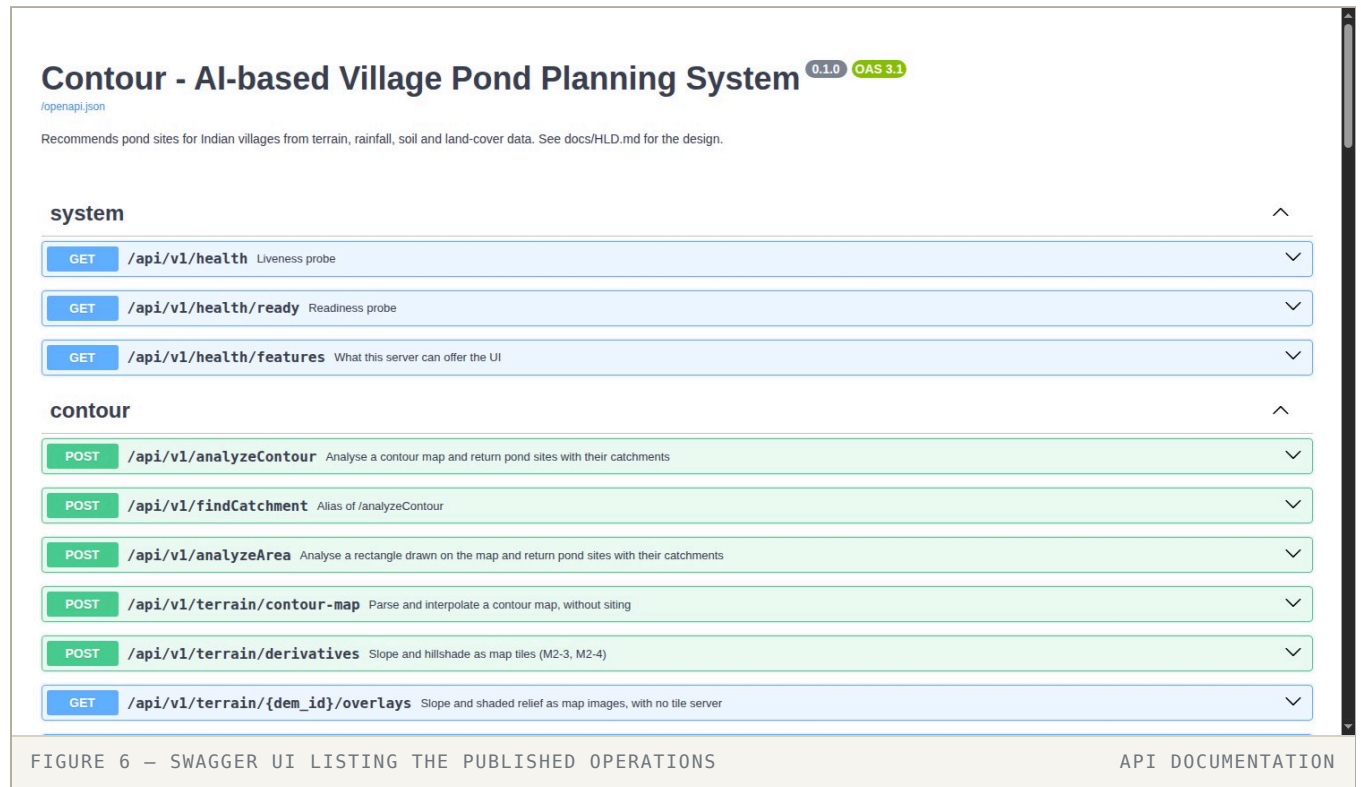
LIMIT	VALUE
Selected area	0.1 to 100 km ²
Contour upload	KML or KMZ, up to 50 MB
At once	One analysis per system, three systems
Beyond that	Queued in the web application; 503 with Retry-After from the API

Measured by scripts/loadtest.py on stu79 sys3 against http://10.1.75.53:3274: three API instances of 512 MB and one CPU each behind the gateway; process memory sampled once a second on each system. Failover: scripts/failover_check.py on stu79 sys4, killing its own API process while a user pinned to it kept making requests through the gateway.

7 · API documentation

The service publishes an OpenAPI 3.1.0 description of all 34 operations, browsable three ways:

FORM	URL	USE
Swagger UI	<code>http://10.1.75.53:3274/docs</code>	Interactive — build and send a request in the page
ReDoc	<code>http://10.1.75.53:3274/redoc</code>	Laid out for reading
Raw spec	<code>http://10.1.75.53:3274/openapi.json</code>	Machine-readable, for client generation



7.1 Every route

Analysis

METHOD	PATH	PURPOSE
POST	<code>/api/v1/analysis</code>	Start an analysis as a background job (M6-1)
POST	<code>/api/v1/analysis/area</code>	Start analysing a rectangle drawn on the map
DELETE	<code>/api/v1/analysis/{job_id}</code>	Abandon a job and drop its record
GET	<code>/api/v1/analysis/{job_id}/result</code>	The finished analysis (M6-5)
GET	<code>/api/v1/analysis/{job_id}/status</code>	Job state, percentage and per-step outcomes (M6-3)

Contour

METHOD	PATH	PURPOSE
POST	/api/v1/analyzeArea	Analyse a rectangle drawn on the map and return pond sites with their catchments
POST	/api/v1/analyzeContour	Analyse a contour map and return pond sites with their catchments
POST	/api/v1/findCatchment	Alias of /analyzeContour
POST	/api/v1/hydrology/catchment	Delineate the catchment above any point (M3-9b, FR-4)
POST	/api/v1/hydrology/streams	Drainage network with Strahler order (M3-3)
POST	/api/v1/land/available	Land parcels a pond could actually be dug on (M5-6, FR-3)
POST	/api/v1/land/cadastral	Upload a cadastral layer and check tenure at each site (M10-3, M10-4, FR-11)
POST	/api/v1/terrain/contour-map	Parse and interpolate a contour map, without siting
GET	/api/v1/terrain/contour-map/{dem_id}/contours	Echo the parsed contours as GeoJSON
POST	/api/v1/terrain/contours	Generate contour lines from a DEM at any interval (M2-5, M2-6)
POST	/api/v1/terrain/derivatives	Slope and hillshade as map tiles (M2-3, M2-4)
GET	/api/v1/terrain/{dem_id}/overlay/{product}	One terrain layer as a PNG (slope or shaded relief)
GET	/api/v1/terrain/{dem_id}/overlays	Slope and shaded relief as map images, with no tile server

Export

METHOD	PATH	PURPOSE
GET	/api/v1/export/{job_id}	Every result layer as one GeoJSON file (M7-4)

Reports

METHOD	PATH	PURPOSE
POST	/api/v1/reports/generate	Render a finished analysis as a PDF report (M7-1, M7-3)
GET	/api/v1/reports/{report_id}/download	Download a rendered report

Suitability

METHOD	PATH	PURPOSE
POST	/api/v1/suitability/analyze	Run the suitability engine as a job, optionally with your own weights (M6-11)
GET	/api/v1/suitability/weights	The AHP weights the model ships with, and their consistency audit (M6-7)
POST	/api/v1/suitability/weights/ahp	Derive criterion weights from a pairwise comparison matrix (M6-7)
GET	/api/v1/suitability/{job_id}/compare	Two to five candidate sites side by side (M10-6, FR-12)
GET	/api/v1/suitability/{job_id}/sites	Ranked candidate sites with per-criterion contributions (M6-11, FR-9)

System

METHOD	PATH	PURPOSE
GET	/api/v1/health	Liveness probe
GET	/api/v1/health/features	What this server can offer the UI
GET	/api/v1/health/ready	Readiness probe

Villages

METHOD	PATH	PURPOSE
GET	/api/v1/villages/resolve	Reverse-geocode a coordinate to the smallest area that contains it
GET	/api/v1/villages/search	Search villages by name, in Latin or Devanagari
GET	/api/v1/villages/{village_id}	Village metadata and canonical identifiers
GET	/api/v1/villages/{village_id}/boundary	The best available boundary, labelled with what it outlines
GET	/api/v1/villages/{village_id}/imagery	Satellite tile template, attribution and framing (FR-1)

7.2 Response shape

A successful analysis returns one object. The blocks a caller is most likely to want:

KEY	HOLDS
summary	The three results for the recommended site: pond location, catchment area, water collected
terrain_source	Where the terrain came from — the uploaded sheet, or Copernicus GLO-30 for a selected area
contour_map	What was read from the file: lines, levels, interval, bounds, chosen CRS (null for a selected area)
interpolated_terrain	Grid size, cell size, interpolation diagnostics
recommended_site	The top-ranked site, expanded — a copy of candidate_sites[0]
candidate_sites[]	Per site: location, score, criteria breakdown, catchment, runoff, pond
candidate_sites[].catchment	metrics, pour_point, snapped, quality, and the boundary as a GeoJSON polygon
suitability	Tier, AHP weights, and the exclusion audit
environment	Which data layers answered, which did not, and why
explanation	Plain-language summary and caveats for each site
warnings[]	Anything the reader should verify before acting

7.3 Errors

Failures use RFC 9457 problem documents, so a client can branch on `type` rather than parse prose:

```
{
  "type": "/errors/validation",
  "title": "Validation failed",
  "status": 422,
  "detail": "file is not well-formed XML/KML: syntax error: line 1, column 0",
  "trace_id": "c2c91fb78f7c"
}
```

STATUS	WHEN
400	Missing field, or a filename that is not a contour map
404	Unknown <code>dem_id</code> or job id — analyses are kept 24 hours
413	Upload above the size limit
422	The file parsed but cannot be analysed — no contours, no elevations, a KMZ that expands too far
503	A required dependency is down; the response names it

8 · AI usage

Claude (Anthropic) was used as a coding assistant and to prepare this document.